

AGRIAI 2.0 | FORENSIC SOFTWARE EXAMINATION

Remediation and Production Operations

A technical plan to move from resilient prototype to controlled production service

Volume 09

Prepared and authored solely by Thaddeus Nii Teiko Tagoe

Founder, designer and sole creator of AgriAI

Computer Science, University of Ghana

Repository examined: ThaddeusHackz/agriai-v2

Evidence commit: 991457272b7fec548e02c645c5593b930af5ea29

Examination date: 18 August 2026

Classification: Internal technical record

Technical basis

This document is grounded in direct source review, route enumeration, dependency audit, secret pattern scanning, strict TypeScript checking, ESLint, production compilation, PostgreSQL integration tests and local HTTP behavior tests. Live third party AI accounts were not available in the examination environment, so provider success is assessed from code paths and contracts rather than a claim of live production availability.

1. Remediation principle

Do not remove graceful degradation. Preserve the provider ladders while adding identity, quotas, schemas, data isolation and evidence. Reliability and security must be designed together. A blocked abusive request should be cheap, and a provider failure should remain understandable to the farmer.

2. P0 actions before public deployment

Order	Action	Acceptance test
P0.1	Delete known password fallback and fail secure at startup	Boot without ADMIN_PASSWORD fails before listening; known password never authenticates
P0.2	Remove or authorize /api/history	A second browser cannot read, write or delete a chat by identifier
P0.3	Upgrade Next.js and lockfile	npm audit has no high or critical finding and all tests pass
P0.4	Edge limits on all AI, media, health and write routes	Burst tests return 429 before a provider call
P0.5	Make provider health private and cached	Anonymous health never calls Gemini and never returns provider text

3. Secure bootstrap pattern

```
At process start:
  require ADMIN_EMAIL
  require ADMIN_PASSWORD length >= 16
  reject known, placeholder and breached values
  if users table is empty:
    create first admin once
  else:
    ignore seed password

On password change:
  update bcrypt hash
  revoke every session for that user
  write audit event
```

4. Rate limit design

Route group	Suggested starting control	Additional budget
chat	10/min/device, 30/hour/IP	Max concurrent 2, daily token ceiling
vision	3/min/device, 20/day/IP	Decoded byte and image dimension limits
generate	1/min/device, 10/day/account	Queue and provider cost ceiling
transcribe	2/min/device, 30/day	Duration and decoded byte limits
tts	5/min/device, 100/day	Character budget, private no-store
search and live prices	10/min/IP	Cache normalized query results
contact, subscribe, feedback	5/hour/IP	CAPTCHA or proof after threshold
health	Platform allowlist	No model call in liveness

Use a shared edge or Redis-backed limiter, not an in-process Map. Derive client IP only from a trusted platform header. Return Retry-After. Record a low-cardinality rejection metric without retaining unnecessary personal data.

5. History ownership design

```
Option A, simplest:
  delete /api/history because Chat.tsx does not use it

Option B, retained browser history:
  server creates random history token
  token stored in HttpOnly SameSite cookie
  database stores hash(token), not raw token
  every GET, POST and DELETE verifies hash ownership
  POST accepts append-only typed messages, not arbitrary record replacement
  rotate token and apply retention
```

6. Runtime validation

Adopt one schema library for every route. Define string lengths, numeric bounds, enums, arrays and discriminated response types once. Reject unknown nested settings fields. Validate settings.chat.temperature from 0 to 2, maxTokens to a safe provider range, model against an approved list, and prompts to an audited size. Validate decoded media rather than only encoded text length.

7. Browser hardening

```
Recommended response policy outline:
  Content-Security-Policy with per-request nonce
  frame-ancestors 'none'
  object-src 'none'
  base-uri 'self'
  form-action 'self'
  connect-src 'self' plus only required browser origins
  Permissions-Policy: microphone=(self), camera=(), geolocation=()
  Strict-Transport-Security at HTTPS edge
  X-Content-Type-Options: nosniff
  Referrer-Policy: strict-origin-when-cross-origin
  poweredByHeader: false
```

8. Prompt and agronomic safety

- * Version every system prompt and record who approved it.
- * Keep retrieved web content outside the highest-trust instruction layer.
- * Allowlist trusted Ghana agriculture, MoFA, COCOBOD and research domains for high-risk advice.
- * Require a human-confirmation warning for pesticide, fertilizer, veterinary, financial and emergency recommendations.
- * Build test sets for English, Twi, Ga, Ewe, Hausa and French with human reviewers.
- * Measure disease sensitivity, specificity, calibration and not-a-plant rejection by crop and image condition.
- * Never present model-generated confidence as measured accuracy without calibration evidence.

9. Persistence redesign

Current	Target	Migration approach
One JSONB document	Normalized settings, users, sessions, chats, messages and PII tables	Dual write, verify counts, cut reads, remove mirror
Raw session token	Hashed token with indexed expiry	Invalidate current sessions during migration
Full object save	Row-level transactions	Repository methods per entity
No revision	updated_at and optimistic version	Reject stale admin writes
No retention	Scheduled deletion and aggregation	Policy by entity with audit receipts
No audit table	Append-only admin_events	Record actor, request, target and hashes

10. Role policy

Role	Allow	Deny by default
viewer	Read aggregate analytics	PII, settings, exports and mutation
content editor	Public copy, approved knowledge drafts and prices	Users, sessions, chats, subscribers, prompt activation, deletes
agronomist	Review knowledge and prompt versions	User admin, infrastructure and raw PII
administrator	Users, role assignment, approved settings and operations	Direct secret reads
owner	Critical reset, key rotation and policy	No silent action, require reauthentication and audit

11. Observability

- * Assign a request ID at the edge and include it in provider and persistence logs.
- * Measure request count, latency, status, provider, fallback tier, token or byte use and 429 rate.
- * Never log provider keys, passwords, raw cookies, full chat text or media.
- * Alert on login failure spikes, owner actions, AI spend threshold, high fallback rate, DB flush failure and document growth.
- * Use synthetic checks for local liveness, authenticated readiness and provider checks at controlled intervals.

12. CI gate

```
npm ci
npm run typecheck
npm run lint -- --max-warnings=0
npm run test:unit
npm run test:api
npm run test:e2e
npm run build
npm audit --audit-level=high
secret scan
software bill of materials generation
container image scan
```

13. Required route tests

- * Default credentials rejected and startup fails secure.
- * Session expiry, logout, rotation, role changes and password reset revoke access.
- * Editor cannot call every denied server endpoint directly.
- * CSRF origin and token controls reject cross-site mutation.
- * History ownership prevents cross-browser access.
- * Rate limits stop provider calls and return Retry-After.
- * SSE handles model success, partial failure, Cloudflare fallback, local fallback and abort.
- * Malformed media and oversized decoded content are rejected before provider calls.
- * Database writes remain correct under concurrent instances and retryable provider latency.

14. Deployment checklist

1. Use a unique secret store and rotate any key ever exposed to logs or local files.
2. Verify patched dependency and immutable lockfile.
3. Set production admin bootstrap through a one-time procedure.

4. Enable shared rate limits and provider budget alerts.
5. Run migrations and encrypted backup restore test.
6. Apply CSP, HSTS, frame and permissions policy.
7. Run API, browser, load and security tests against staging.
8. Verify Ghana language and agronomy review sign-off.
9. Document incident owner, key rotation, session revocation and rollback.
10. Deploy gradually and watch provider, fallback, error and cost metrics.

15. Final engineering position

AgriAI already has a strong resilience idea: every important farmer path should end with an understandable result. Production maturity requires the same discipline for hostile input, identity, concurrency, privacy and evidence. The remediation plan keeps the core experience while converting implicit assumptions into enforced controls.

Authorship and control statement

This volume was prepared, authored and issued solely under the name of Thaddeus Nii Teiko Tagoe. This plan gives Thaddeus Tagoe a direct implementation order for securing and scaling the system he created. The repository records identify Thaddeus Tagoe as the founder, sole designer, developer and maintainer of AgriAI.